

Técnicas e Algoritmos em Ciência de Dados

Tarefa 3

Este trabalho deve ser enviado até o dia 20 de maio de 2026, às 09h.
Envios atrasados serão penalizados em 10% por hora de atraso.

Tópicos avaliados

Esta tarefa oferece uma oportunidade emocionante para os alunos colocarem em prática seus conhecimentos adquiridos em sala de aula, usando redes neurais para resolver problemas do mundo real, tanto na classificação quanto na regressão. Os alunos aplicarão os conceitos que aprenderam para construir, treinar e otimizar redes neurais, usando um conjunto de validação para ajustar hiperparâmetros. Os alunos também se acostumarão a gerar gráficos importantes durante o treinamento para analisar o comportamento dos modelos. Ao final do projeto, os alunos terão adquirido experiência prática na implementação de redes neurais.

Instruções

Identificador

Por favor, use o **número aleatório de 6 dígitos que você usou para as TAREFAS 1,2** e escreva-o na primeira célula do notebook. Certifique-se de manter uma cópia desse número, pois ele será usado para fornecer o feedback.

Submissão

Envie seus arquivos através do ECLASS. Os arquivos enviados não podem ser substituídos por ninguém e não podem ser lidos por nenhum outro aluno. Você pode, no entanto, substituir seu envio quantas vezes quiser, reenviando, embora apenas a última versão enviada seja mantida.

Se você tiver problemas na última hora, envie um e-mail como anexo para alberto.paccanaro@fgv.br com o assunto "URGENTE – ENVIO DE TAREFA 3 ". NO corpo da mensagem, explique o motivo de não enviar através do ECLASS.

IMPORTANTE

- Seu envio consistirá em um único bloco de anotações Python implementando suas soluções.
- **O nome do arquivo será o número aleatório que o identifica (por exemplo, 568423.ipynb)**
- Este curso é composto por 2 partes. Certifique-se de que o código de ambas as partes está colocado nas células de código relevantes no notebook.
- **NÃO ENVIE NENHUM CONJUNTO DE DADOS**, apenas o código.
- Qualquer função auxiliar que você usará deve ser incluída no notebook – não envie scripts adicionais.
- *Por favor, tente não usar ferramentas de IA como ChatGPT, Copilot, Gemini, etc. Este trabalho foi elaborado para que você se familiarize com conceitos importantes que só podem ser aprendidos por meio da experiência prática.*

Todo o trabalho que você enviar deve ser exclusivamente seu próprio trabalho. As submissões de trabalhos do curso serão verificadas para isso.
--

Caso seja suspeitada alguma atividade irregular nas tarefas (por exemplo, plágio, uso de IA etc.), os laboratórios serão também avaliados e, para cada laboratório considerado inadequado, o aluno perderá o ponto correspondente.

Critérios de marcação

Esta tarefa é obrigatória e vale 10% da sua nota final total para este curso. Para obter notas máximas para cada pergunta, você deve respondê-la corretamente, mas também completamente. Daremos pontos para códigos bem estruturados

IMPORTANTE: Em geral, você pode usar *numpy* ou outras bibliotecas básicas para operações de matriz, mas você não pode usar qualquer função de biblioteca que implementaria alguns dos algoritmos que você é obrigado a implementar. Se você está em dúvida sobre uma função específica, envie-nos um e-mail.

TAREFA

Parte 1 – Regressão com Redes Neurais (valor: 50%)

Baixe do ECLASS

- Tarefa_3_template.ipynb
- student_performance.csv

Conjunto de dados e descrição do problema

Neste exercício, você usará o conjunto de dados *Student Academic Performance - 500 Students*. Este conjunto de dados contém informações relacionadas ao desempenho acadêmico de 500 estudantes e será utilizado para construir modelos de aprendizado de máquina capazes de analisar padrões associados ao rendimento estudantil.

O objetivo principal deste exercício é explorar como diferentes características dos estudantes podem estar relacionadas ao seu desempenho acadêmico. Para isso, você deverá preparar e analisar os dados, além de construir uma rede neural para prever o desempenho dos estudantes.

Este conjunto de dados é útil para praticar conceitos importantes de aprendizado de máquina, como tratamento de variáveis categóricas e numéricas, divisão dos dados em treino e teste, treinamento de modelos, avaliação de desempenho e interpretação dos resultados.

Descrição do Exercício: Predição de desempenho de estudantes com Redes Neurais. Neste exercício, você usará o conjunto de dados *Student Academic Performance - 500 Students*. Você construirá e treinará uma rede neural para prever a pontuação final dos estudantes, representada pela coluna ***final_score*** no conjunto de dados.

Para concluir este exercício, você escreverá código para criar e treinar redes neurais para esse problema:

1. Divida o conjunto de dados em conjuntos de treinamento, validação e teste, usando uma proporção de 70:15:15.
2. Use numpy, para construir uma rede neural que tome os *features* dos estudantes como entrada e preveja a nota final como saída. Você escolherá o número de neurônios por camada e o número de camadas, mas cada camada deverá ter o mesmo número de neurônios. Esses dois valores devem ser parâmetros de entrada para sua rede neural. Ou seja, você não pode codificar cada camada de forma fixa, o que significa que você terá que escrever código que possa trabalhar com um número variável de camadas e neurônios.
3. Codifique os algoritmos de forward e back propagation para aprender os pesos da rede neural. Use o conjunto de treinamento para treinar a rede neural e atualizar os pesos

usando três estratégias mostradas nas aulas passadas: *batch*, *SGD*, e *mini-batch*. Para as camadas ocultas, use a função de ativação sigmoideal. Para evitar overfitting, use *early stopping* durante o treinamento.

4. Monitore o treinamento plotando os erros de treinamento e validação ao longo do treinamento.

O desempenho da sua rede neural poderá variar dependendo do número de camadas, do número de neurônios por camada e da estratégia utilizada para atualização dos pesos.

Para escolher a melhor configuração de rede e avaliar seu desempenho, você:

1. Escolherá 3 possíveis valores do número de camadas ocultas (por exemplo, de 1 a 3) e 3 diferentes valores de neurônios por camada (por exemplo, 100, 200 e 300), mas você também pode escolher valores diferentes.
2. Calculará a perda para cada configuração no conjunto de validação.
3. Gerará 3 *heatmaps*, um para cada estratégia (*batch*, *SGD*, *mini-batch*). Mostrando a perda no conjunto de validação ao plotar o número de camadas e o número de neurônios em uma grade. (Isso ajudará você a visualizar a melhor configuração para a rede neural.)
4. Treinará seu modelo final selecionando a melhor combinação de hyperparameters e avaliará o desempenho final da rede neural usando o *test set* e o *Root Mean Squared Error (RMSE)* como métrica. Em seguida, relatará os resultados obtidos.
5. Treinará sua implementação de linear regression das tarefas anteriores e um *baseline model* feito com TensorFlow, utilizando o training set. Além disso, avaliará o performance da sua rede neural e dos baseline models no test set.

Importante:

- Treine por 50 epochs.
- Defina a taxa de aprendizagem η como 0,01.

Parte 2 – Classificação com Redes Neurais (valor: 50%)

Faça o download dos dados do ECLASS

- `dry_bean_database.csv`

Descrição do conjunto de dados

Este é um conjunto de dados do domínio da biologia. Descreve o problema de classificação de diferentes tipos de feijões secos a partir de características extraídas de imagens. Cada dado

corresponde a um grão de feijão representado por um conjunto de 16 *features* relacionadas à sua forma e dimensão, como área, perímetro, comprimento dos eixos, excentricidade, arredondamento e compacidade. O rótulo representa a classe do feijão, que pode pertencer a uma das 7 variedades: Seker, Barbunya, Bombay, Cali, Dermosan, Horoz ou Sira.

Descrição do exercício: Dry Bean — Classificação multiclasse com Redes Neurais. Neste exercício, você construirá e treinará uma rede neural para prever a variedade de cada grão de feijão usando as 16 características do conjunto de dados. O objetivo é resolver um problema de classificação multiclasse, no qual cada exemplo pertence a uma única classe entre as 7 classes disponíveis.

Para completar este exercício, siga estes passos:

1. Carregue o conjunto de dados, faça a limpeza eliminando duplicados, e faça tratamento das *features* (alguns precisam ser convertidas). Além disso, divida-o em conjuntos de treinamento, validação e teste, usando uma proporção de 70:15:15.
2. Padronize os *features* removendo a média e escalonando para variância unitária. Para fazer isso, realize o seguinte para cada feature (coluna) no conjunto de dados:
 - a. Calcule a média e o desvio padrão em todo o conjunto de treinamento para esse feature.
 - b. Subtraia a média de cada valor nesse feature e divida pelo desvio padrão.
 - c. Aplique a mesma transformação aos conjuntos de validação e teste usando a média e o desvio padrão calculados a partir do conjunto de treinamento.

Observação: *you precisa codificar esta parte, não é permitido usar o scikit-learn.*

A normalização dos features é importante para redes neurais porque:

- *Garante que todas as características tenham a mesma escala, evitando que certas características dominem o processo de aprendizagem devido à sua maior magnitude.*
 - *Melhora a estabilidade numérica do processo de treinamento, tornando a rede neural menos sensível à escolha da taxa de aprendizado e outros hiperparâmetros.*
3. Usando NumPy construa uma rede neural que recebe os *features* como entrada e prevê as classes. Você escolherá o número de neurônios por camada e o número de camadas. Você fornecerá essas informações como uma lista de entrada, onde o comprimento da lista determina o número de camadas ocultas e cada elemento é o número de neurônios dessa camada oculta. Por exemplo, uma lista `layers = [64, 128, 256]` deveria produzir uma rede com 4 camadas, sendo 3 camadas ocultas com 64, 128 e 256 neurônios cada.

4. Codifique o algoritmo de forward and back propagation para aprender os pesos da rede neural. Use o conjunto de treinamento para treinar a rede neural e atualizar os pesos usando três estratégias mostradas nas aulas passadas: *batch*, *SGD*, e *mini-batch*. Para as camadas ocultas, use a função de ativação sigmoide. Para evitar o *overfitting*, usará *early stopping*.
5. Monitore o treinamento traçando as perdas de treinamento e validação ao longo do treinamento.

O desempenho da sua rede neural será diferente dependendo do número de camadas, número de neurônios por camada e três estratégias mostradas nas aulas passadas: *batch*, *SGD*, e *mini-batch*.

Para escolher a melhor configuração de rede e avaliar seu desempenho, você:

1. Para cada estratégia de atualização dos pesos, selecione 3 configurações diferentes de camada (note que neste exercício o número de neurônios por camada não precisa ser o mesmo para cada camada).
2. Calcule o loss (erro) para cada configuração no conjunto de validação.
4. No final deste processo você terá 9 valores de perda (um para cada configuração). Gerará 3 *heatmaps*, um para cada estratégia (*batch*, *SGD*, *mini-batch*). Mostrando a perda no conjunto de validação ao plotar o número de camadas e o número de neurônios em uma grade. (Isso ajudará você a visualizar a melhor configuração para a rede neural.)
3. Treine seu modelo final selecionando a melhor combinação de hiperparâmetros e avalie o desempenho final da rede neural usando o conjunto de teste e calculando a área sob a curva ROC, a precisão, o score F1, e a acurácia como métricas e relate isso. Note que este não é um problema de classificação binária, portanto, essas medidas precisam ser calculadas considerando cada classe como positiva em relação a todas as outras como negativas e, em seguida, calculando a média.
4. Treinará sua implementação de *logistic regression* das tarefas anteriores e um *baseline model* feito com TensorFlow, utilizando o training set, e calculará o desempenho usando o test set. Além disso, avaliará o performance da sua rede neural e dos baseline models, utilizando *barplots*.

Importante:

- Treine por 1000 epochs.
- Defina a taxa de aprendizagem η como 0,01.